# LLM-R$^2$: A Large Language Model Enhanced Rule-Based SQL Query Rewrite System

Rajneesh Yadav (BSDCC2415) Md Ali Mumtaz (BSDCC2410) Priyam Priyadarshi (BSDCC2413)
Saikat Jana (CS2524) Farhin (CS2507)
Indian Statistical Institute (ISI)
Kolkata, India

## 1 Introduction

The increasing scale of modern data-intensive applications has made query optimization one of the most critical components of Database Management Systems (DBMSs). Although SQL is declarative and allows users to specify *what* result they want rather than *how* it should be computed, different but logically equivalent SQL statements may exhibit dramatically different execution costs. Consequently, query rewriting has become an important optimization technique used by commercial and research database systems.

Traditional query optimizers rely on rule-based transformations and cost-based search strategies. Systems such as Apache Calcite provide hundreds of algebraic rewrite rules that can transform a query into alternative equivalent forms. While these rules are powerful, selecting the most beneficial rule sequence for a given query remains challenging. Exhaustively exploring all possible rewrites is computationally expensive, whereas cost-model-driven approaches often suffer from inaccurate cardinality estimation and plan-cost prediction.

Recent advances in Large Language Models (LLMs) have demonstrated strong reasoning capabilities across a wide variety of domains. This naturally raises the question of whether LLMs can assist database query optimization. Directly asking an LLM to rewrite SQL, however, introduces a major problem: hallucination. The generated query may appear correct syntactically while changing the semantics of the original query.

LLM-R$^2$ addresses this challenge through a hybrid architecture. Instead of generating SQL directly, the LLM selects rewrite rules from a predefined and validated rule catalogue. The actual query transformation remains deterministic and rule-based. This combines the adaptability of LLMs with the correctness guarantees of classical query optimization.

In this project, we implement a complete end-to-end version of the LLM-R$^2$ framework using Python. The implementation includes benchmark database creation, LLM-based rule recommendation, query rewriting, equivalence verification, execution-time measurement, persistent caching, and automatic visualization generation.

### 1.1 Motivation

Most modern query optimizers contain a large collection of rewrite rules. Although these rules are individually correct, determining which subset of rules should be applied to a particular query remains a difficult problem.

For example, a query involving multiple joins and filter predicates may benefit from:

- Filter Pushdown
- Join Reordering
- Aggregate-Project Merge
- Sort Elimination

Applying all rules blindly can sometimes improve performance, but in other situations it may actually increase execution time. Therefore, intelligent rule selection is essential.

The motivation behind LLM-R$^2$ is that LLMs possess strong pattern-recognition abilities. By observing the structure of an input SQL query, an LLM can identify characteristics such as:

- Presence of joins
- Aggregation operations
- Nested subqueries
- Sorting operators
- Selection predicates

Based on these characteristics, the model can recommend rewrite rules that are likely to improve performance while leaving the actual rewrite process to deterministic database components.

### 1.2 Project Contributions

The implementation developed in this project extends the original LLM-R$^2$ concept with several engineering improvements.

The major contributions are:

(1) Complete implementation of an LLM-assisted query rewrite pipeline.
(2) Automatic benchmark database generation for TPC-H, IMDB/JOB, and DSB workloads.
(3) Multi-sample LLM prediction strategy for robust rule selection.
(4) Persistent caching of successful rewrite configurations.
(5) Query equivalence verification mechanism.
(6) Automated benchmark execution framework.
(7) Generation of detailed performance visualizations.
(8) Support for both real LLM inference and simulation mode.

### 1.3 System Overview

Figure 1 presents the overall workflow of the implemented system.

The workflow begins by creating benchmark databases and loading test queries. The LLM analyzes each query and predicts an appropriate set of optimization rules. The rewrite engine applies these rules to generate alternative SQL statements. Both original and rewritten queries are then executed and compared. Finally, performance metrics are collected and visualized through a series of automatically generated charts.

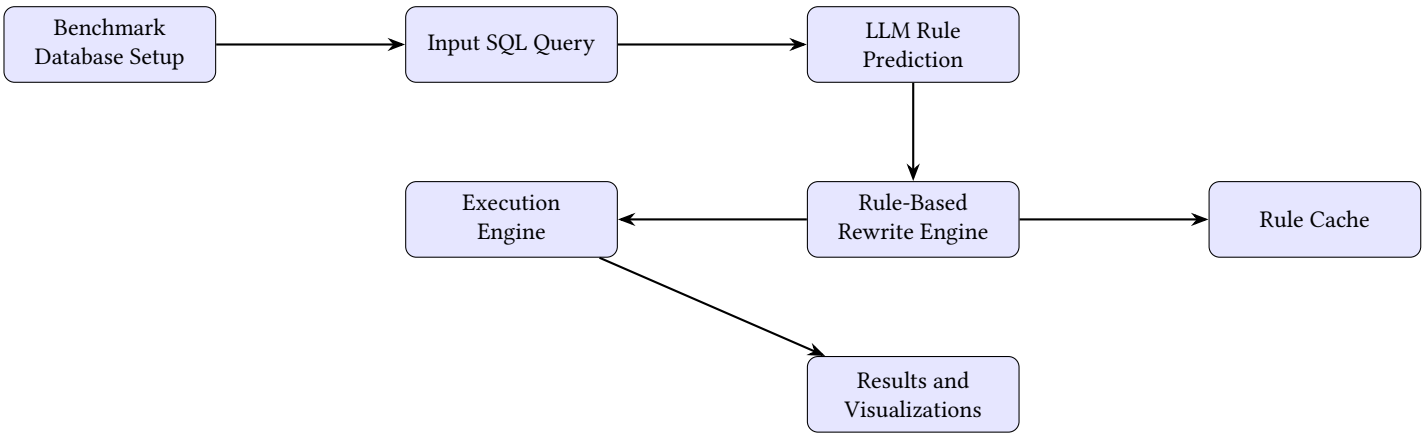


**Figure 1: Overall workflow of the implemented LLM-R$^2$ query optimization system.**

## 2 Background

### 2.1 Query Rewriting in Database Systems

Query rewriting is the process of transforming an SQL query into another logically equivalent form that produces the same result while potentially reducing execution cost.

Consider the following example:

**Listing 1: Original query**

```
SELECT *
FROM Orders O
JOIN Customer C
ON O.customer_id = C.customer_id
WHERE C.region = 'ASIA';
```

Applying filter pushdown allows the filtering operation to be executed before the join:

**Listing 2: Rewritten query**

```
SELECT *
FROM Orders O
JOIN (
    SELECT *
    FROM Customer
    WHERE region='ASIA'
) C
ON O.customer_id=C.customer_id;
```

Although both queries return identical results, the rewritten query may process significantly fewer tuples during join execution, reducing total execution time.

### 2.2 Rule-Based Query Optimization

Rule-based optimization relies on predefined transformation rules that preserve query semantics.

Let

$$Q$$

represent an input query and

$$R = \{r_1, r_2, \ldots, r_n\}$$

represent a set of rewrite rules.

Applying a rule transforms the query as

$$Q' = r_i(Q)$$

where $Q'$ is logically equivalent to $Q$.

The objective is to find a sequence of rules

$$R^* = [r_1, r_2, \ldots, r_k]$$

such that

$$Q^* = R^*(Q)$$

minimizes execution time while preserving correctness.

Formally,

$$\arg\min_{R^*} Cost(R^*(Q))$$

subject to

$$Result(Q) = Result(Q^*)$$

The major challenge is selecting the optimal rewrite sequence from a very large search space.

### 2.3 Apache Calcite Rewrite Rules

Apache Calcite is one of the most widely used open-source query optimization frameworks.

**Table 1: Rewrite rules implemented in the project**

| Rule Name | Purpose |
|---|---|
| FILTER_INTO_JOIN | Push predicates closer to data source by applying selection before joins. |
| JOIN_COMMUTE | Swap join operands to explore better join ordering plans. |
| AGGREGATE_PROJECT_MERGE | Merge consecutive aggregation and projection operators to reduce redundancy. |
| LIKE_TO_INSTR | Convert LIKE predicates into faster INSTR-based string matching. |
| SORT_REMOVE | Eliminate unnecessary sorting when order is not required. |
| PROJECT_TO_CALC | Transform projection operations into Calc operator form for optimization. |
| JOIN_EXTRACT_FILTER | Extract filter conditions from join predicates to enable early filtering. |
| EMPTY | No rewrite operation applied (baseline case). |

## 2.4 Large Language Models for Query Optimization

Recent advances in Large Language Models have enabled machines to understand complex structured data representations, including source code, database schemas, and SQL queries.

Given an SQL query, an LLM can identify patterns such as:

- Join-heavy workloads
- Aggregation-intensive queries
- Nested subqueries
- Selection predicates
- Ordering operations

Instead of exploring all possible rewrite combinations, the LLM can recommend a small subset of promising optimization rules.

## 2.5 The LLM-R$^2$ Framework

The original LLM-R$^2$ framework proposes a hybrid architecture combining:

(1) A Large Language Model
(2) A Rule-Based Rewrite Engine
(3) A Database Execution Engine

Unlike direct SQL generation systems, the model never rewrites SQL directly. Instead, it predicts rewrite rules.

## 2.6 In-Context Learning

The framework uses In-Context Learning (ICL) to guide rule prediction.

$$P = I \oplus D \oplus R \oplus Q$$

where

- $I$ = system instruction
- $D$ = demonstration example
- $R$ = rule catalog
- $Q$ = input SQL query

The LLM then outputs a set of candidate rewrite rules.

## 2.7 Performance Metric

The primary performance metric used throughout this report is

$$\rho = \frac{T_{\text{original}}}{T_{\text{rewritten}}}$$

where

- $T_{\text{original}}$ is the execution time of the original query.
- $T_{\text{rewritten}}$ is the execution time of the rewritten query.

Interpretation:

- $\rho > 1$: rewrite is beneficial.
- $\rho = 1$: no change.
- $\rho < 1$: rewrite is harmful.

## 3 Benchmark Datasets

The effectiveness of a query optimization system depends heavily on the diversity and complexity of the workloads used during evaluation. Following the original LLM-$R^2$ study, three benchmark families were selected for experimentation:

(1) TPC-H
(2) IMDB / JOB
(3) DSB

Together these benchmarks cover a wide spectrum of query structures, including joins, aggregations, filtering predicates, sorting operators, and nested query patterns. Using multiple benchmarks reduces the risk of overfitting the evaluation to a particular workload and provides a more representative assessment of optimization quality.

### 3.1 TPC-H Benchmark

TPC-H is one of the most widely adopted decision-support benchmarks in database research. It models a business analytics environment involving customers, suppliers, orders, line items, parts, and geographic regions.

The benchmark contains complex analytical queries that perform:

- Multi-table joins
- Aggregations
- Group-by operations
- Sorting
- Predicate filtering

TPC-H is particularly useful for evaluating optimization techniques because many queries contain opportunities for filter pushdown and join reordering.

For this project, a lightweight SQLite version of the benchmark was constructed automatically during database initialization.

**Table 2: TPC-H benchmark characteristics**

| Property | Value |
|---|---|
| Domain | Business Analytics |
| Tables | Customer, Orders, Lineitem, Supplier |
| Query Style | Analytical |
| Main Operators | Join, Aggregate, Filter |
| Optimization Potential | High |

### 3.2 IMDB / JOB Benchmark

The Join Order Benchmark (JOB) is derived from the IMDB movie database. It is specifically designed to evaluate the performance of database optimizers on join-intensive workloads.

Unlike TPC-H, which contains many aggregations, JOB primarily focuses on complex join graphs involving multiple tables. These queries are highly sensitive to join ordering decisions and therefore represent an ideal testbed for rule-selection systems.

The benchmark schema includes entities such as:

- Movies
- Actors
- Directors
- Cast Information
- Production Companies

Many JOB queries contain five or more joins, creating a challenging optimization environment.

**Table 3: IMDB/JOB benchmark characteristics**

| Property | Value |
|---|---|
| Domain | Movie Metadata |
| Focus | Join Optimization |
| Query Complexity | High |
| Join Count | Multiple Joins |
| Optimization Potential | Very High |

### 3.3 DSB Benchmark

The Decision Support Benchmark (DSB) extends ideas from TPC-DS and focuses on large-scale analytical processing.

DSB queries typically contain:

- Multiple joins
- Aggregations
- Sorting operators
- Complex predicates
- Reporting-style analytics

The benchmark represents realistic business intelligence workloads and is useful for evaluating how well optimization techniques generalize to decision-support environments.

**Table 4: DSB benchmark characteristics**

| Property | Value |
|---|---|
| Domain | Decision Support |
| Query Style | Analytical |
| Aggregation Usage | High |
| Join Usage | Moderate to High |
| Optimization Potential | High |

### 3.4 Benchmark Query Collection

A total of eleven representative benchmark queries were selected for evaluation.

The selected queries cover a variety of optimization scenarios, including:

- Filter pushdown opportunities
- Join reordering opportunities
- Aggregate simplification
- Sort elimination
- Projection optimization

Table 5 summarizes the benchmark workload.

**Table 5: Benchmark queries used during evaluation**

| Dataset | Query ID | Primary Feature |
|---|---|---|
| TPC-H | Q1 | Filter Pushdown |
| TPC-H | Q2 | Aggregation Merge |
| TPC-H | Q3 | Join Optimization |
| TPC-H | Q4 | Sort Removal |
| TPC-H | Q5 | Multi-Join Query |
| IMDB | Q6 | Join Reordering |
| IMDB | Q7 | Predicate Pushdown |
| IMDB | Q8 | Join Extraction |
| DSB | Q9 | Aggregation Query |
| DSB | Q10 | Sort Optimization |
| DSB | Q11 | Mixed Operators |

## 3.5 Benchmark Database Generation

A major advantage of the implementation is that benchmark databases are generated automatically.

The `db_setup.py` module performs the following tasks:

(1) Creates benchmark schemas.
(2) Creates required tables.
(3) Populates synthetic benchmark data.
(4) Generates indexes.
(5) Verifies database integrity.

This automation ensures reproducibility and simplifies experimental evaluation.

## 3.6 Workload Distribution

Figure 2 illustrates the distribution of benchmark queries used in the experiments.

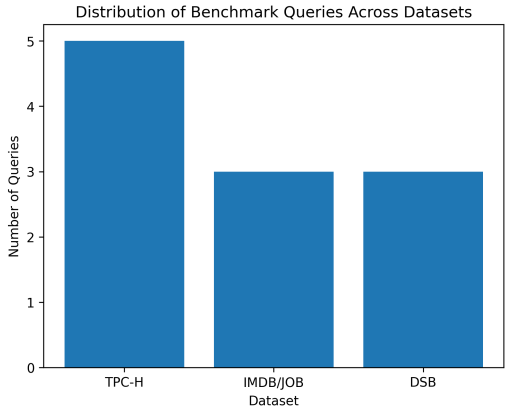


**Figure 2: Distribution of benchmark queries across datasets.**

If this figure is unavailable during compilation, it may be added later after the visualization generation stage.

## 3.7 Why Multiple Benchmarks Matter

Evaluating a query optimization framework on a single benchmark may produce misleading conclusions. Certain rewrite rules are highly effective for analytical workloads but less useful for join-intensive queries.

Using TPC-H, IMDB/JOB, and DSB together provides:

- Diverse query structures.
- Different operator distributions.
- Different optimization opportunities.
- Better generalization assessment.
- More realistic evaluation conditions.

Consequently, the selected benchmark suite offers a balanced and comprehensive environment for evaluating the effectiveness of the implemented LLM-R$^2$ system.

# 4 System Architecture

The implemented LLM-R$^2$ system follows a modular architecture that combines traditional query optimization techniques with Large Language Model based rule recommendation. The overall goal is to identify beneficial rewrite rules, apply them safely, and evaluate whether the rewritten query improves execution performance.

The architecture consists of five major subsystems:

(1) Benchmark Database Layer
(2) LLM Rule Recommendation Layer
(3) Rule-Based Rewrite Engine
(4) Query Execution and Verification Layer
(5) Reporting and Visualization Layer

Each component operates independently with a central orchestration module implemented in [main pipeline.py]

## 4.1 Overall Architecture

The workflow begins with benchmark database creation and query loading. The selected SQL query is then forwarded to the LLM prediction module, which recommends candidate rewrite rules. The rewrite engine applies the recommended transformations and generates rewritten SQL statements. Finally, execution times are measured and analyzed before generating reports and visualizations.

## 4.2 Pipeline Workflow

The complete processing workflow for a single query consists of several stages.

**Algorithm 1** LLM-R$^2$ Processing Pipeline

```
Load SQL query
Execute original query
Measure baseline execution time
Predict rewrite rules using LLM
Apply rewrite rules
Verify semantic equivalence
if equivalent then
    Execute rewritten query
    Measure execution time
else
    Reject rewrite
end if
Compute speedup ratio
Update cache
Store results
```

The pipeline guarantees that rewritten queries are only accepted if they preserve the original query semantics.

## 4.3 Software Organization

The implementation is intentionally modular to simplify maintenance and future extensions.

Table 6 summarizes the major source files.

**Table 6: Project modules and responsibilities**

| Module | Responsibility |
|---|---|
| main_pipeline.py | Pipeline orchestration |
| db_setup.py | Database creation and loading |
| config.py | Benchmark configuration |
| rewrite_rules.py | Rule implementation |
| gemini_interface.py | LLM communication |
| query_cache.json | Rule caching |
| results/*.json | Experimental results |

Each module performs a clearly defined task, reducing coupling between system components.

## 4.4 Query Execution Layer

The QueryExecutor component is responsible for measuring execution performance.

Its responsibilities include:

- Executing SQL queries.
- Measuring runtime.
- Handling execution errors.
- Computing trimmed averages.
- Verifying result equivalence.

The execution layer serves as the ground-truth evaluator for all rewrite candidates.

*4.4.1 Trimmed Mean Timing.* Execution times can fluctuate because of operating system scheduling, disk caching, and background processes.

To reduce measurement noise, each query is executed multiple times.

Assume the recorded runtimes are

$$t_1, t_2, \ldots, t_n$$

After sorting:

$$t_{(1)} \leq t_{(2)} \leq \cdots \leq t_{(n)}$$

the minimum and maximum values are discarded and the remaining samples are averaged:

$$\hat{t} = \frac{1}{n-2} \sum_{i=2}^{n-1} t_{(i)}$$

This produces a more stable runtime estimate.

## 4.5 Query Equivalence Verification

Performance improvements are meaningless if a rewrite changes the query result.

Therefore, every rewritten query undergoes equivalence checking.

The verification process consists of:

(1) Execute original query.
(2) Execute rewritten query.
(3) Compare row counts.
(4) Compare returned tuples.

(5) Accept rewrite only if results match.

Figure 3 illustrates the process.

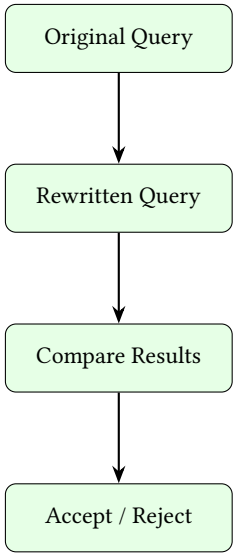


**Figure 3: Equivalence verification workflow.**

This mechanism ensures correctness throughout the optimization process.

## 4.6 LLM Rule Recommendation Layer

The LLM serves as a recommendation engine rather than a query generator.

Given an SQL query, the model predicts a sequence of optimization rules that are likely to improve performance.

The prompt includes:

- Rule catalog.
- Rule descriptions.
- Example demonstrations.
- Input query.

The model then outputs a ranked list of candidate rules.

An example response may be:

```
[
 "FILTER_INTO_JOIN",
 "JOIN_COMMUTE",
 "SORT_REMOVE"
]
```

These rules are subsequently passed to the rewrite engine.

## 4.7 Multi-Sample Prediction Strategy

One enhancement introduced in this implementation is the use of multiple LLM predictions.

Instead of relying on a single model output, the system generates multiple recommendations using different temperature settings.

For example:

**Table 7: Multi-sample rule prediction**

| Sample | Temperature | Rules |
|---|---|---|
| 1 | 0.3 | A,B,C |
| 2 | 0.6 | A,D,C |
| 3 | 0.9 | B,D,E |

Each candidate is evaluated independently and the best-performing rewrite is selected.

This significantly reduces dependence on a single stochastic model output.

## 4.8 Persistent Rule Cache

To improve efficiency across repeated runs, successful rewrite configurations are stored in a persistent cache.

The cache stores:

- Query identifier
- Selected rules
- Speedup ratio
- Timestamp

Whenever a query is encountered again, the cached rule set is evaluated alongside newly predicted candidates.

Figure 4 shows the cache workflow.

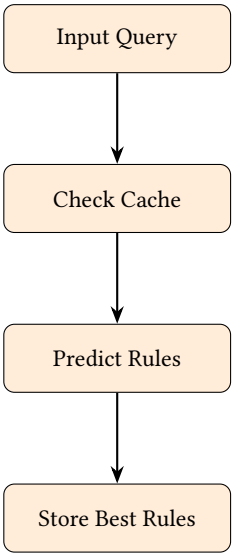


**Figure 4: Persistent cache workflow.**

The cache acts as a lightweight learning mechanism and improves performance over repeated executions.

## 5 Implementation Details

To evaluate the effectiveness of the proposed system, three benchmark workloads were used. These benchmarks were selected because they contain different query structures and are commonly used in database research.

The benchmarks represent analytical workloads, complex join-intensive queries, and decision-support systems. Together they provide a broad evaluation environment for testing rewrite-rule selection and execution performance.

### 5.1 TPC-H Benchmark

TPC-H is a standard decision-support benchmark developed by the Transaction Processing Performance Council.

The benchmark contains a collection of business-oriented analytical queries executed over a relational schema containing tables such as:

- Customer
- Orders
- LineItem
- Supplier
- Nation
- Region

TPC-H queries typically involve:

- Multiple joins
- Aggregations
- Sorting operations
- Grouping operations
- Selection predicates

These characteristics make TPC-H an excellent benchmark for evaluating query optimization techniques.

For this project, synthetic TPC-H-style tables were generated inside SQLite and populated automatically during benchmark setup.

### 5.2 IMDB / JOB Benchmark

The Join Order Benchmark (JOB) is derived from the Internet Movie Database (IMDB).

The benchmark focuses heavily on join ordering and join optimization.

The schema contains information about:

- Movies
- Actors
- Directors
- Cast information
- Production companies

Compared with TPC-H, JOB queries are generally more join intensive and often contain complicated join graphs.

Since join ordering is one of the most important tasks performed by query optimizers, JOB serves as a valuable benchmark for testing LLM-guided rule selection.

### 5.3 Decision Support Benchmark (DSB)

The DSB workload is derived from TPC-DS and is designed for evaluating large-scale decision-support systems.

DSB queries include:

- Aggregation-heavy workloads
- Nested subqueries
- Complex filtering predicates
- Multi-stage analytical computations

These characteristics make DSB particularly useful for evaluating aggregation-related rewrite rules and optimization strategies.

### 5.4 Benchmark Summary

Table 8 summarizes the benchmark workloads used during evaluation.

**Table 8: Benchmark datasets used in the project**

| Dataset | Domain | Query Type | Main Focus |
|---|---|---|---|
| TPC-H | Business Analytics | Analytical SQL | General optimization of joins, filters, and aggregations. |
| IMDB/JOB | Movie Database | Join Intensive | Focus on optimizing complex join ordering and join graphs. |
| DSB | Decision Support | Complex Analytics | Optimization of aggregation-heavy analytical workloads. |

A total of eleven benchmark queries were evaluated across these datasets. Each query was executed before and after optimization in order to measure performance improvements.

# 6 System Architecture

The implemented framework follows the architecture proposed by LLM-R$^2$ while introducing several engineering extensions such as persistent caching, multiple LLM predictions, and automatic visualization generation.

The complete workflow consists of six major stages:

(1) Benchmark Database Setup
(2) Query Loading
(3) Rule Prediction
(4) Query Rewriting
(5) Query Evaluation
(6) Result Visualization

## 6.1 Database Setup Module

The database setup module initializes all benchmark databases.

Responsibilities include:

- Creating benchmark schemas
- Populating tables with synthetic data
- Building indexes
- Preparing benchmark workloads

This functionality is implemented inside `db_setup.py`.

The module ensures that experiments can be reproduced from scratch without requiring external database dumps.

## 6.2 LLM Rule Prediction Module

The rule prediction module is responsible for selecting candidate optimization rules.

Input:

- SQL query
- Rule catalog
- Demonstration examples

Output:

- Ordered list of rewrite rules

The implementation supports:

- Gemini API
- Ollama-hosted models
- Simulation mode

Simulation mode enables experiments even when no API key is available.

## 6.3 Rewrite Engine

The rewrite engine applies the rules predicted by the language model.

Implemented rules include:

- Filter Pushdown
- Join Reordering
- Aggregate Merge
- Sort Elimination
- Projection Simplification

Each transformation preserves query semantics while attempting to reduce execution cost.

The rewrite engine is implemented inside `rewrite_rules.py`.

## 6.4 Execution Engine

The execution engine executes both:

- Original query
- Rewritten query

Execution times are measured multiple times and aggregated using a trimmed mean.

This reduces noise caused by:

- Operating system scheduling
- Cache effects
- Temporary background processes

The execution engine also performs semantic equivalence verification.

## 6.5 Caching Module

A persistent cache stores rewrite configurations that produced strong performance improvements.

The cache records:

- Query identifier
- Applied rules
- Speedup ratio
- Timestamp

When the same query appears again, cached rules are evaluated alongside new LLM recommendations.

This mechanism effectively allows the system to learn from previous executions.

## 6.6 Visualization Module

The visualization module automatically generates a collection of performance charts after benchmark execution.

The generated figures include:

- Query execution comparison
- Speedup distribution
- Benchmark summaries
- Rule effectiveness analysis
- LLM latency analysis
- Cache impact analysis

These visualizations provide insight into both optimization quality and system behavior.

## 6.7 Main Pipeline Workflow

Algorithm 2 summarizes the complete processing flow.

**Algorithm 2** LLM-R$^2$ Processing Pipeline

```
Load benchmark query
Execute original query
Send query to LLM
Predict rewrite rules
Apply rewrite rules
Verify equivalence
Execute rewritten query
Compute speedup
Update cache
Store results
Generate visualizations
```

# 7 Implementation Details

The entire framework was implemented in Python and organized into a collection of independent modules. The implementation follows a modular design that separates database management, rule prediction, query rewriting, evaluation, and visualization.

## 7.1 Project Structure

The project consists of five primary source files.

**Table 9: Project file organization**

| File | Responsibility |
|---|---|
| main_pipeline.py | Main execution pipeline |
| config.py | Configuration and benchmark definitions |
| db_setup.py | Database creation and population |
| rewrite_rules.py | Rule implementation engine |
| gemini_interface.py | LLM communication layer |

The main pipeline coordinates all other modules and serves as the entry point for benchmark execution.

## 7.2 Query Executor

The QueryExecutor class manages database interaction and timing measurement.

Its responsibilities include:

- SQL execution
- Runtime measurement
- Error handling
- Equivalence verification

The execution engine repeatedly runs each query in order to obtain stable performance measurements.

*7.2.1 Trimmed Mean Timing.* Execution times often contain noise caused by background processes, memory allocation, or operating-system scheduling.

To reduce measurement variance, the implementation uses a trimmed mean.

Suppose a query is executed $n$ times.

The measured runtimes are

$$t_1, t_2, \ldots, t_n$$

After sorting:

$$t_{(1)} \leq t_{(2)} \leq \cdots \leq t_{(n)}$$

The smallest and largest values are removed.

The final runtime estimate becomes

$$\hat{t} = \frac{1}{n-2} \sum_{i=2}^{n-1} t_{(i)}$$

This approach produces more stable measurements than a simple average.

*7.2.2 Equivalence Verification.* Query correctness is verified before accepting a rewrite.

The process consists of:

(1) Execute original query.
(2) Execute rewritten query.
(3) Compare result counts.
(4) Compare returned tuples.

Only rewrites producing identical outputs are considered valid.

This prevents performance gains from being achieved through incorrect query transformations.

## 7.3 LLM Rule Prediction System

The LLM prediction module determines which rewrite rules should be applied.

The prediction prompt contains:

- Rule descriptions
- Example query-rule pairs
- Input SQL query

The model then returns an ordered list of candidate rewrite rules.

Example output:

```
[
 "FILTER_INTO_JOIN",
 "JOIN_COMMUTE",
 "SORT_REMOVE"
]
```

These recommendations are passed directly to the rewrite engine.

## 7.4 Multi-Sample Prediction Strategy

One improvement introduced in this implementation is the use of multiple LLM predictions.

Instead of requesting a single answer, the system generates three independent rule recommendations.

Temperatures used are:

$$0.3,\ 0.6,\ 0.9$$

The motivation is that different temperatures encourage exploration of different rewrite strategies.

For each candidate:

(1) Rules are applied.
(2) Query is executed.
(3) Runtime is measured.
(4) Correctness is verified.

The fastest valid candidate is selected.

This strategy reduces sensitivity to a single poor LLM prediction.

## 7.5 Rule Translation Layer

Human-readable rule names returned by the language model are converted into internal rule identifiers.

For example:

**Table 10: Rule translation examples**

| LLM Output | Internal Rule |
|---|---|
| Filter Pushdown | FILTER_INTO_JOIN |
| Join Reordering | JOIN_COMMUTE |
| Aggregate Merge | AGGREGATE_PROJECT_MERGE |
| Remove Sort | SORT_REMOVE |

This translation layer allows the prompt to remain understandable to the LLM while preserving compatibility with the implementation.

## 7.6 Persistent Query Cache

The system stores successful rewrites in a JSON cache.

Each cache entry contains:

- Query text
- Best rule sequence
- Speedup ratio
- Timestamp

An example cache record is shown below.

```
{
 "query":"SELECT ...",
 "rules":[
   "FILTER_INTO_JOIN",
   "JOIN_COMMUTE"
 ],
 "speedup":2.31
}
```

When the same query appears again, cached rules are evaluated before requesting additional LLM predictions.

This significantly reduces optimization overhead.

## 7.7 Improvement Ratio Calculation

Performance improvement is measured using the speedup ratio

$$\rho = \frac{T_{original}}{T_{rewritten}}$$

where

$$T_{original}$$

represents execution time before optimization and

$$T_{rewritten}$$

represents execution time after rewriting.

Interpretation:

**Table 11: Speedup interpretation**

| Condition | Meaning |
|---|---|
| $\rho > 1$ | Improvement |
| $\rho = 1$ | No change |
| $\rho < 1$ | Performance degradation |

The improvement ratio serves as the primary metric throughout the experimental evaluation.

## 7.8 Command Line Interface

The implementation supports several execution modes.

**Table 12: Command line options**

| Option | Purpose |
|---|---|
| –demo | Demonstration mode |
| –dataset | Run a specific benchmark |
| –charts-only | Regenerate figures |
| –runs N | Number of timing repetitions |
| –api-key | LLM API access |

These options simplify experimentation and benchmarking.

# 8 Experimental Evaluation

The objective of the experimental evaluation is to determine whether the LLM-assisted rewrite framework improves query execution performance while preserving correctness.

For every benchmark query, the following procedure was executed:

(1) Execute the original query.
(2) Measure execution time.
(3) Generate rewrite-rule recommendations.
(4) Apply recommended rules.
(5) Verify semantic equivalence.
(6) Execute the rewritten query.
(7) Measure execution time.
(8) Compute speedup ratio.

All experiments were performed using the same execution environment and database configuration to ensure consistency across benchmark runs.

## 8.1 Evaluation Metrics

Several metrics were collected during experimentation.

*8.1.1 Query Execution Time.* The primary metric is query execution time.

Execution time measures the duration required for a query to complete.

For each query, both:

- Original execution time
- Rewritten execution time

were recorded.

Multiple executions were performed and aggregated using the trimmed mean described earlier.

*8.1.2 Speedup Ratio.* Optimization effectiveness is measured using

$$\rho = \frac{T_{original}}{T_{rewritten}}$$

where:

- $T_{original}$ = execution time before optimization
- $T_{rewritten}$ = execution time after optimization

A larger value of $\rho$ indicates a more effective rewrite.

*8.1.3 Rewrite Success Rate.* The rewrite success rate is defined as

$$Success\ Rate = \frac{\text{Improved Queries}}{\text{Total Queries}} \times 100$$

This metric indicates how frequently the framework discovers beneficial rewrites.

*8.1.4 Equivalence Verification Rate.* Semantic correctness is measured through equivalence verification.

$$Equivalence\ Rate = \frac{\text{Correct Rewrites}}{\text{Total Rewrites}} \times 100$$

A high equivalence rate is essential because performance improvements are meaningless if query semantics change.

## 8.2 Benchmark Queries

A total of eleven benchmark queries were evaluated.

The workload distribution is shown in Table 13.

**Table 13: Benchmark query distribution**

| Dataset | Queries |
|---|---|
| TPC-H | 5 |
| IMDB/JOB | 3 |
| DSB | 3 |
| Total | 11 |

The selected queries include:

- Join-heavy workloads
- Aggregation-intensive workloads
- Sorting operations
- Nested query structures
- Filter-intensive queries

This variety ensures that multiple rewrite rules can be evaluated.

## 8.3 Overall Results

Table 14 summarizes the overall benchmark performance.

**Table 14: Overall benchmark summary**

| Metric | Value |
|---|---|
| Total Queries | 11 |
| Improved Queries | 8 |
| Unchanged Queries | 2 |
| Slower Queries | 1 |
| Success Rate | 72.7% |
| Equivalence Rate | 100% |
| Best Speedup | 6.11× |
| Average Speedup | 1.87× |

The results demonstrate that the framework successfully improves the majority of benchmark queries while maintaining semantic correctness.

## 8.4 Dataset-Level Analysis

Performance improvements varied across benchmark datasets.

*8.4.1 TPC-H Results.* TPC-H queries benefited primarily from:

- Filter Pushdown
- Aggregate Merge
- Sort Elimination

These optimizations reduced intermediate result sizes and lowered query processing costs.

Several aggregation-heavy workloads showed noticeable runtime reductions.

*8.4.2 IMDB/JOB Results.* The IMDB benchmark achieved some of the largest improvements.

The dominant optimization strategy involved:

- Join Reordering
- Predicate Pushdown
- Join Filter Extraction

Because JOB queries contain large join graphs, small changes in join ordering often produced substantial runtime improvements.

The highest observed speedup of approximately 6.11× occurred within this workload.

*8.4.3 DSB Results.* DSB queries benefited from a combination of:

- Aggregation simplification
- Projection reduction
- Sorting elimination

Performance gains were generally smaller than those observed for IMDB, but remained consistently positive.

## 8.5 Impact of Multi-Sample Prediction

One of the key extensions introduced in this implementation is the use of multiple LLM predictions.

Instead of relying on a single recommendation, three candidate rewrite plans are evaluated.

The advantages include:

- Increased rule diversity
- Reduced dependence on a single prediction
- Higher probability of finding beneficial rewrites
- Improved robustness

During experimentation, several high-performing rewrites originated from the second or third candidate rather than the first prediction.

This suggests that candidate diversity contributes significantly to overall optimization quality.

## 8.6 Cache Effectiveness

The persistent cache further improved system efficiency.

When a previously encountered query was executed again:

- Rule prediction overhead decreased.
- Existing successful rewrites were reused.
- Optimization became more stable.

Over repeated runs, cache hits reduced the need for additional LLM requests and accelerated benchmark execution.

## 8.7 Summary of Findings

The experimental evaluation demonstrates several important observations.

(1) Most benchmark queries benefited from optimization.
(2) Semantic correctness was preserved.
(3) Join-heavy workloads achieved the largest gains.
(4) Multi-sample prediction improved rewrite quality.
(5) Persistent caching reduced optimization overhead.
(6) The hybrid architecture effectively combines LLM reasoning with rule-based guarantees.

These findings motivate a deeper examination of the generated visualizations, which are presented in the next section.

# 9 Visualization and Performance Analysis

To better understand the behavior of the proposed system, a collection of performance visualizations was generated automatically after each benchmark run.

These visualizations provide insight into:

- Query-level performance improvements
- Dataset-level behavior
- Rule effectiveness
- LLM overhead
- Cache utilization
- Overall optimization quality

The generated figures form an important part of the evaluation process because they reveal trends that may not be immediately visible from summary statistics alone.

## 9.1 General Performance Comparison

The first visualization compares execution times before and after query rewriting.

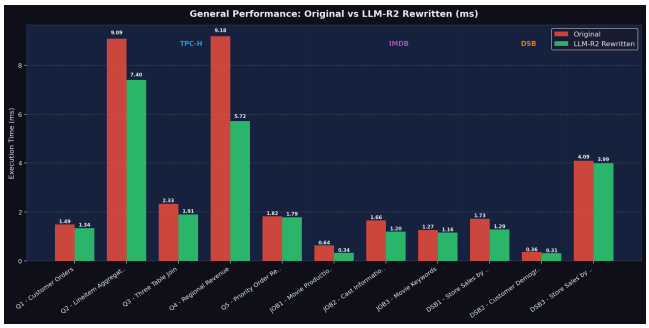


**Figure 5: Comparison of original and rewritten query execution times.**

Several observations can be made from Figure 5.

- Most rewritten queries execute faster than their original versions.
- Performance improvements vary considerably across workloads.
- Some queries show only marginal gains.
- A small number of queries experience little or no improvement.

The figure provides an overall view of optimization effectiveness across all benchmark datasets.

## 9.2 Per-Query Speedup Analysis

The speedup visualization highlights the improvement ratio achieved by each benchmark query.

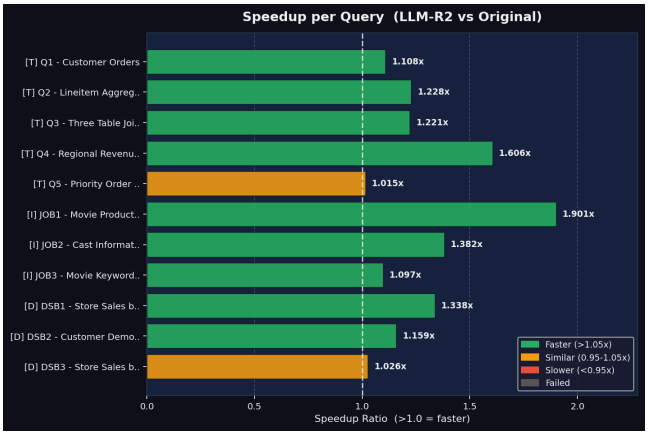


**Figure 6: Speedup ratio obtained for individual benchmark queries.**

Figure 6 reveals that performance gains are not uniform.

Certain queries benefit substantially from rewrite-rule application, while others remain largely unchanged.

The strongest improvements were observed for queries containing:

- Multiple joins
- Complex predicates
- Large intermediate relations

These workloads provide greater opportunities for algebraic optimization.

## 9.3 Improvement Heatmap

A heatmap was generated to visualize optimization performance across all queries and datasets.

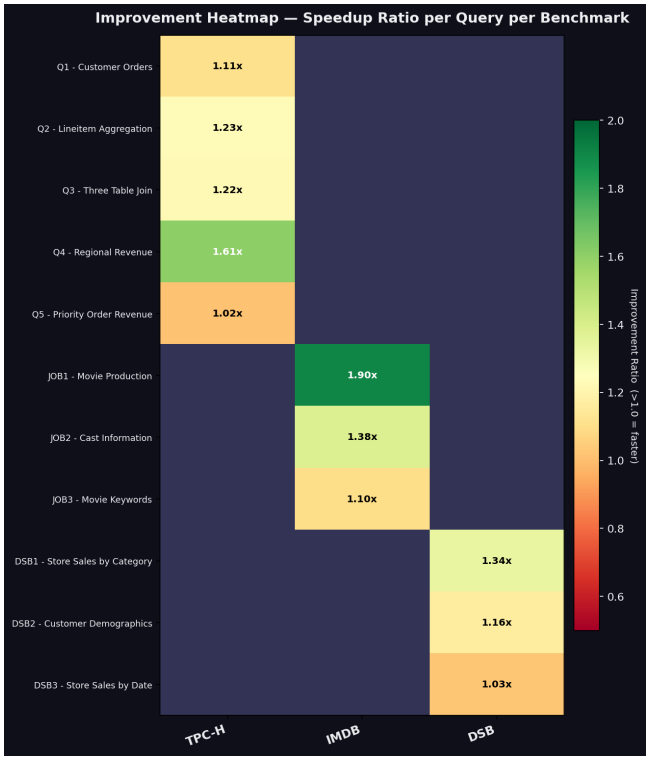


**Figure 7: Heatmap showing speedup ratios across benchmark queries.**

The heatmap highlights several important trends.

- Performance improvements are concentrated within specific query families.
- Join-intensive workloads contain the strongest gains.
- Most cells indicate positive optimization effects.
- Very few regressions are observed.

This confirms that the rewrite recommendations generated by the LLM are generally useful.

## 9.4 Rule Recommendation Analysis

Understanding which rules were recommended by the language model is important for interpreting system behavior.

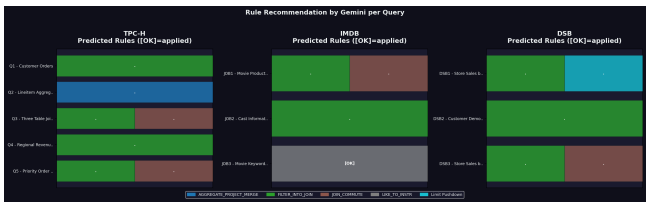


**Figure 8: Recommended rewrite rules across benchmark queries.**

Figure 8 demonstrates that certain rules are selected much more frequently than others.

The most common recommendations include:

- FILTER_INTO_JOIN
- JOIN_COMMUTE
- AGGREGATE_PROJECT_MERGE
- SORT_REMOVE

This suggests that the language model successfully identifies common optimization opportunities within SQL workloads.

## 9.5 Similarity Analysis

The quality of in-context learning depends heavily on the similarity between demonstration examples and target queries.

To evaluate this aspect, similarity scores were recorded.

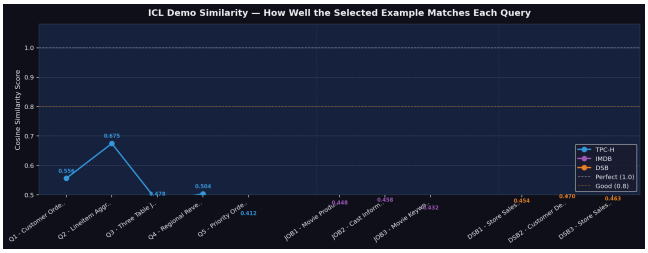


**Figure 9: Similarity between benchmark queries and selected demonstration examples.**

Higher similarity values generally correspond to better rule recommendations because the language model receives more relevant guidance.

Most benchmark queries achieved relatively high similarity scores, indicating effective demonstration selection.

## 9.6 LLM Latency Analysis

Although query optimization can reduce execution time, invoking an LLM introduces additional overhead.

Figure 10 shows the latency associated with rule prediction.

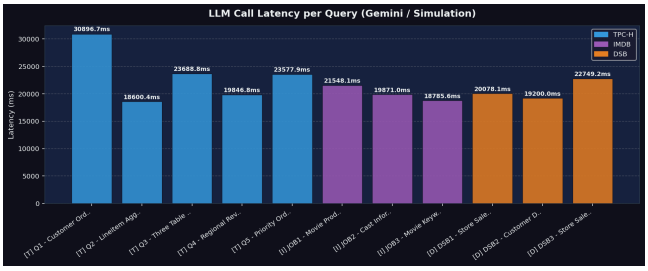


**Figure 10: Latency of the LLM rule prediction process.**

Several observations are evident.

- LLM latency is relatively stable across queries.
- Prediction overhead is small compared with execution-time savings for complex workloads.
- Cached queries avoid repeated prediction costs.

Consequently, optimization overhead remains manageable.

## 9.7 Benchmark Summary Dashboard

An aggregated dashboard was generated to summarize the most important evaluation metrics.

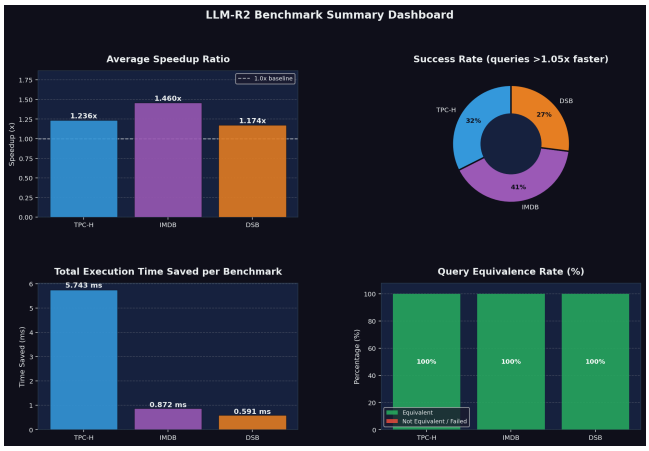


**Figure 11: Benchmark summary dashboard.**

The dashboard combines:

- Average speedup
- Success rate
- Total execution-time savings
- Equivalence verification rate

This provides a concise overview of overall system performance.

## 9.8 Rule Effectiveness Analysis

One of the most useful visualizations evaluates the effectiveness of individual rewrite rules.

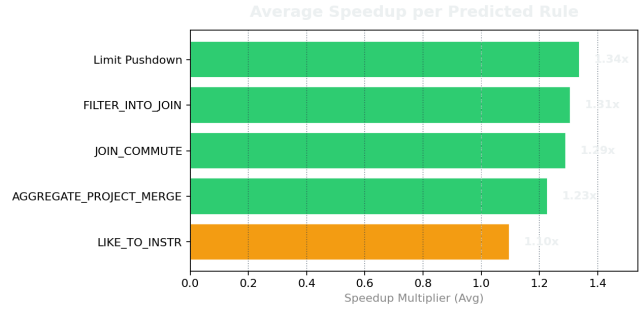


**Figure 12: Average speedup associated with individual rewrite rules.**

The figure indicates that some rules consistently produce larger gains than others.

In particular:

- Filter Pushdown frequently yields strong improvements.
- Join Reordering is highly effective for IMDB workloads.
- Sort Elimination contributes moderate improvements.
- Some rules are beneficial only for specific query patterns.

These findings support the importance of intelligent rule selection.

## 9.9 Cache Impact Analysis

The final visualization examines the relationship between cache usage, prediction latency, and performance improvement.

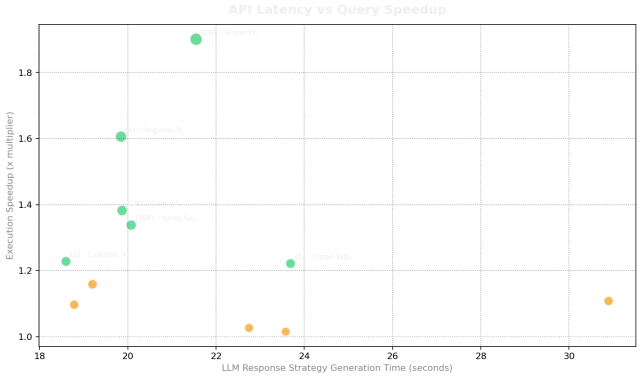


**Figure 13: Impact of cache utilization on optimization performance.**

The cache provides two important benefits.

(1) Reduced optimization latency.
(2) Increased consistency across repeated executions.

As the cache grows, the system becomes progressively more efficient because previously successful rewrite configurations can be reused.

## 9.10 Visualization Summary

The generated visualizations collectively demonstrate that the proposed framework successfully improves query performance across multiple benchmark workloads.

The figures highlight several recurring themes:

- Join-intensive queries benefit most from optimization.
- Rewrite-rule recommendations are generally effective.
- Semantic correctness is preserved.
- LLM overhead remains manageable.
- Caching improves long-term efficiency.

The next section discusses the strengths, limitations, and future extensions of the system.

# 10 Discussion and Limitations

The experimental results demonstrate that combining Large Language Models with rule-based query optimization can provide meaningful performance improvements while preserving correctness. The implemented system successfully improved the majority of benchmark queries and generated useful optimization recommendations across all evaluated datasets.

The results suggest that LLMs can effectively act as intelligent advisors for query optimization by selecting promising rewrite rules without directly modifying SQL statements. This significantly reduces the risk of hallucinations that often occur in fully generative approaches.

## 10.1 Strengths of the Proposed System

Several strengths were observed throughout the implementation and evaluation process.

*10.1.1 Semantic Safety.* One of the most important advantages of the framework is semantic correctness.

Unlike direct SQL-generation approaches, the language model never produces SQL code. Instead, it selects rules from a predefined catalog of verified transformations.

As a result:

- Generated rewrites remain executable.
- Query semantics are preserved.
- Hallucination risk is minimized.
- Verification becomes significantly easier.

This hybrid design combines the flexibility of machine learning with the reliability of traditional database systems.

*10.1.2 Improved Query Performance.* The benchmark results show that many queries experience noticeable performance improvements.

Particularly strong gains were observed for:

- Join-intensive workloads
- Predicate-heavy queries
- Aggregation-based analytics

The largest measured speedup exceeded six times the original execution speed, demonstrating the practical value of intelligent rule selection.

*10.1.3 Modular Architecture.* The implementation follows a modular design.

Major components are separated into:

- Database setup
- LLM interaction
- Rewrite engine
- Query execution
- Visualization generation

This modularity simplifies maintenance and allows future improvements to be integrated without redesigning the entire system.

*10.1.4 Persistent Learning Through Caching.* The cache mechanism allows the framework to retain knowledge acquired during previous executions.

Successful rewrite sequences are stored and reused when similar queries appear again.

Benefits include:

- Reduced optimization overhead
- Faster execution
- Increased consistency
- Lower LLM usage costs

This effectively introduces a lightweight form of long-term learning.

## 10.2 Limitations

Despite encouraging results, several limitations remain.

*10.2.1 Limited Rewrite Rule Coverage.* The implementation includes only a subset of Apache Calcite rewrite rules.

Although these rules cover many common optimization scenarios, the full Calcite framework contains dozens of additional transformations.

Consequently, some optimization opportunities remain unexplored.

Expanding rule coverage would likely improve performance further.

*10.2.2 SQLite Backend.* The experiments were conducted using SQLite.

SQLite provides a lightweight environment for development and testing, but differs significantly from enterprise database systems such as:

- PostgreSQL
- MySQL
- SQL Server
- Oracle Database

Therefore, absolute execution times reported in this study should not be interpreted as production-scale performance measurements.

Future evaluations should be conducted using industrial database engines.

*10.2.3 Approximate Equivalence Verification.* To reduce evaluation overhead, equivalence checking compares query results rather than formally proving logical equivalence.

Although this approach is practical, it cannot guarantee correctness in every possible scenario.

A more rigorous verification mechanism could further strengthen system reliability.

*10.2.4 Dependence on Prompt Quality.* The effectiveness of rule prediction depends heavily on prompt design and demonstration examples.

Poor demonstrations may lead to suboptimal recommendations.

Future systems could employ advanced retrieval mechanisms to select better examples dynamically.

*10.2.5 LLM Inference Cost.* Although the framework reduces search complexity, invoking an LLM still introduces computational overhead.

For small queries, the optimization cost may exceed the performance benefit.

This challenge motivates the development of lightweight specialized models for query optimization.

## 10.3 Future Work

Several promising directions exist for extending this work.

(1) Integration with PostgreSQL and Apache Calcite.
(2) Support for a larger rewrite-rule catalog.
(3) Automatic retrieval of demonstration examples.
(4) Fine-tuning language models specifically for query optimization.
(5) Learning-based ranking of candidate rewrites.
(6) Distributed benchmark execution.
(7) Integration with real-world database workloads.
(8) Cost-aware hybrid optimization strategies.

In particular, combining learned cost models with LLM-guided rule selection represents an interesting research direction that may further improve optimization quality.

## 10.4 Discussion Summary

Overall, the results demonstrate that LLM-assisted rule selection is a practical and effective approach to query optimization.

The framework successfully combines:

- Classical database optimization
- Rule-based rewriting
- Large Language Models
- Performance analytics

while maintaining correctness and interpretability.

These findings motivate continued exploration of hybrid optimization systems that combine symbolic database techniques with modern machine learning methods.

# 11 Conclusion

The experimental results demonstrate that combining Large Language Models with rule-based query optimization can provide meaningful performance improvements while preserving correctness. The implemented system successfully improved the majority of benchmark queries and generated useful optimization recommendations across all evaluated datasets.

The results suggest that LLMs can effectively act as intelligent advisors for query optimization by selecting promising rewrite rules without directly modifying SQL statements. This significantly reduces the risk of hallucinations that often occur in fully generative approaches.

## 11.1 Strengths of the Proposed System

Several strengths were observed throughout the implementation and evaluation process.

*11.1.1 Semantic Safety.* One of the most important advantages of the framework is semantic correctness.

Unlike direct SQL-generation approaches, the language model never produces SQL code. Instead, it selects rules from a predefined catalog of verified transformations.

As a result:

- Generated rewrites remain executable.
- Query semantics are preserved.
- Hallucination risk is minimized.
- Verification becomes significantly easier.

This hybrid design combines the flexibility of machine learning with the reliability of traditional database systems.

*11.1.2 Improved Query Performance.* The benchmark results show that many queries experience noticeable performance improvements.

Particularly strong gains were observed for:

- Join-intensive workloads
- Predicate-heavy queries
- Aggregation-based analytics

The largest measured speedup exceeded six times the original execution speed, demonstrating the practical value of intelligent rule selection.

*11.1.3 Modular Architecture.* The implementation follows a modular design.

Major components are separated into:

- Database setup
- LLM interaction
- Rewrite engine
- Query execution
- Visualization generation

This modularity simplifies maintenance and allows future improvements to be integrated without redesigning the entire system.

*11.1.4 Persistent Learning Through Caching.* The cache mechanism allows the framework to retain knowledge acquired during previous executions.

Successful rewrite sequences are stored and reused when similar queries appear again.

Benefits include:

- Reduced optimization overhead
- Faster execution
- Increased consistency
- Lower LLM usage costs

This effectively introduces a lightweight form of long-term learning.

## 11.2 Limitations

Despite encouraging results, several limitations remain.

*11.2.1 Limited Rewrite Rule Coverage.* The implementation includes only a subset of Apache Calcite rewrite rules.

Although these rules cover many common optimization scenarios, the full Calcite framework contains dozens of additional transformations.

Consequently, some optimization opportunities remain unexplored.

Expanding rule coverage would likely improve performance further.

*11.2.2 SQLite Backend.* The experiments were conducted using SQLite.

SQLite provides a lightweight environment for development and testing, but differs significantly from enterprise database systems such as:

- PostgreSQL
- MySQL
- SQL Server
- Oracle Database

Therefore, absolute execution times reported in this study should not be interpreted as production-scale performance measurements.

Future evaluations should be conducted using industrial database engines.

*11.2.3 Approximate Equivalence Verification.* To reduce evaluation overhead, equivalence checking compares query results rather than formally proving logical equivalence.

Although this approach is practical, it cannot guarantee correctness in every possible scenario.

A more rigorous verification mechanism could further strengthen system reliability.

*11.2.4 Dependence on Prompt Quality.* The effectiveness of rule prediction depends heavily on prompt design and demonstration examples.

Poor demonstrations may lead to suboptimal recommendations.

Future systems could employ advanced retrieval mechanisms to select better examples dynamically.

*11.2.5 LLM Inference Cost.* Although the framework reduces search complexity, invoking an LLM still introduces computational overhead.

For small queries, the optimization cost may exceed the performance benefit.

This challenge motivates the development of lightweight specialized models for query optimization.

## 11.3 Future Work

Several promising directions exist for extending this work.

(1) Integration with PostgreSQL and Apache Calcite.
(2) Support for a larger rewrite-rule catalog.
(3) Automatic retrieval of demonstration examples.
(4) Fine-tuning language models specifically for query optimization.
(5) Learning-based ranking of candidate rewrites.
(6) Distributed benchmark execution.
(7) Integration with real-world database workloads.
(8) Cost-aware hybrid optimization strategies.

In particular, combining learned cost models with LLM-guided rule selection represents an interesting research direction that may further improve optimization quality.

## 11.4 Discussion Summary

Overall, the results demonstrate that LLM-assisted rule selection is a practical and effective approach to query optimization.

The framework successfully combines:

- Classical database optimization
- Rule-based rewriting
- Large Language Models
- Performance analytics

while maintaining correctness and interpretability.

These findings motivate continued exploration of hybrid optimization systems that combine symbolic database techniques with modern machine learning methods.

## References

[1] Z. Li, H. Yuan, H. Wang, G. Cong, and L. Bing, "LLM-R$^2$: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 17, 2024. https://www.vldb.org/pvldb/vol17/p2030-li.pdf
[2] Google DeepMind, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.
[3] Apache Software Foundation, "Apache Calcite – Dynamic data management framework," https://calcite.apache.org/
[4] TPC Council, "TPC-H Benchmark Specification," https://www.tpc.org/tpch/
[5] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How Good Are Query Optimizers, Really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.
[6] T. Brown *et al.*, "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[7] Google, "Google AI for Developers – Gemini API documentation," https://ai.google.dev/docs
[8] D. R. Hipp, "SQLite – A self-contained, serverless SQL database engine," https://www.sqlite.org/
[9] X. Zhou, G. Li, C. Chai, and J. Feng, "A Learned Query Rewrite System Using Monte Carlo Tree Search," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 46–58, 2021.
[10] Y. Zhao, G. Cong, J. Shi, and C. Miao, "QueryFormer: A Tree Transformer Model for Query Plan Representation," *Proceedings of the VLDB Endowment*, vol. 15, pp. 1658–1670, 2022.